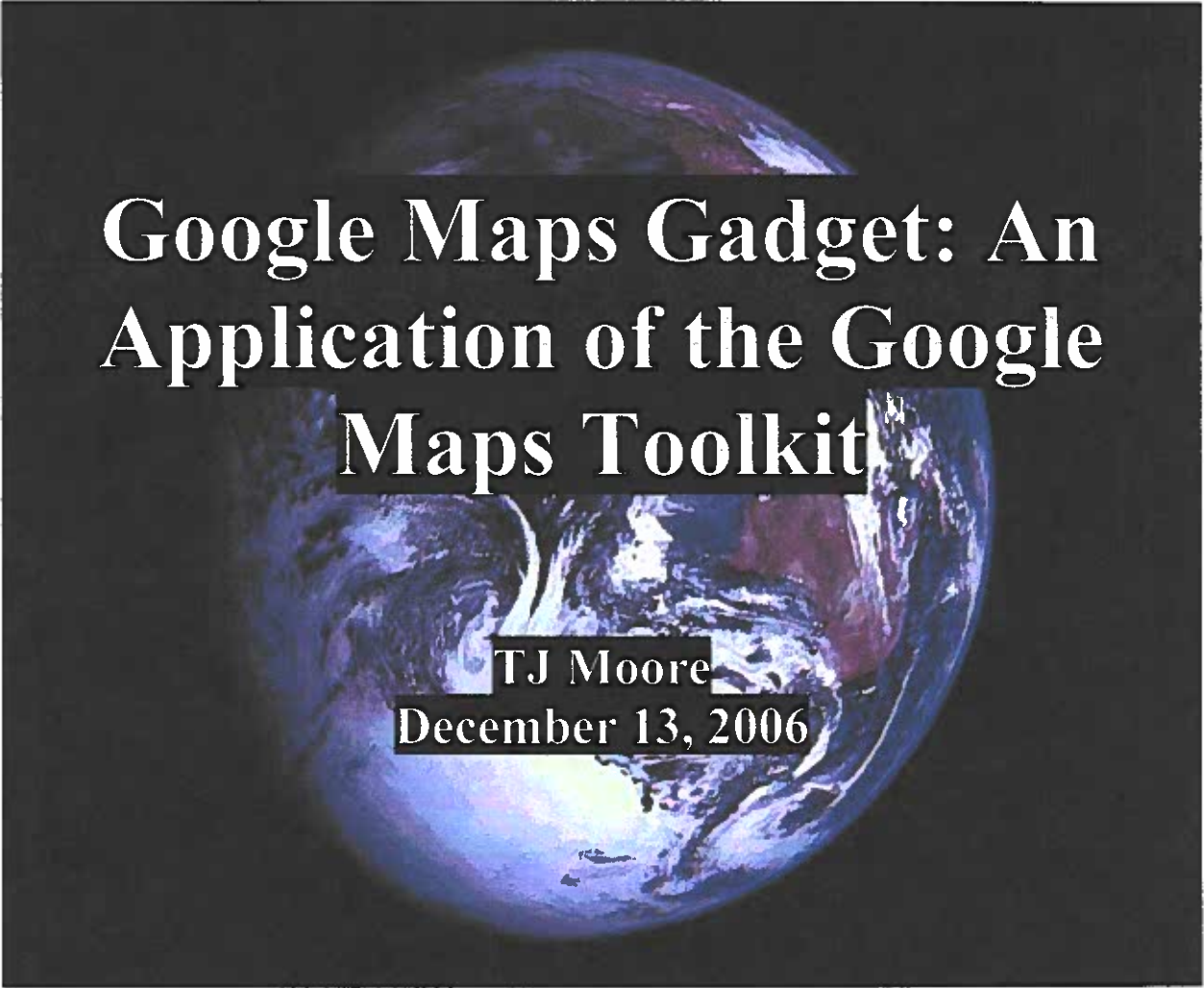




Google Maps Gadget: An Application of the Google Maps Toolkit

TJ Moore

December 13, 2006

**Critical Infrastructure Protection Center
SPAWAR Systems Center Charleston**

P.O. Box 190022
Building 3454 Code 70J
North Charleston, SC 29419-9022



December 13, 2006

***This document is not for distribution
or reproduction as it is the exclusive
property of the U.S. Government
CIPC Charleston.***

Google Maps Gadget:
An Application of the Google Maps Toolkit
Senior Project
December 13, 2006
TJ Moore

Problem Definition

This project was designed to solve two main problems. First of all, this project is meant to give non-technical users the ability to work with many of the powerful features presented in the Google Maps Application Programming Interface (API). [1] The capabilities presented in the Google Maps API allow a web developer to display markings and information on top of satellite imagery, traditional maps or a hybrid of the two. Creative uses of this technology can increasingly be found on many websites throughout the internet. With such a wide range of uses for this type of mapping technology, it would be a great waste if only those people who are able to write their own JavaScript and HTML are able to make use of this tool. Therefore, the first goal of my project is to allow anyone who is able to use a computer and access the internet to be able to make use of this new technology.

The second purpose of this project was to give me the opportunity to become familiar with various network programming tools and procedures. During my time at Charleston Southern University, I have learned a great deal about programming techniques and computer-related topics, but I had not taken many classes that focused on web development. In February 2006, I started an internship at SPAWAR, working in the Critical Infrastructure Protection Center (CIPC). Since the majority of the development done in the CIPC is web based, I felt the need to increase my knowledge of the techniques used for web development.

In mid-June of 2006, Michael Harper, my project lead at the CIPC, suggested I start looking into what could be done with the Google Maps API, so I began learning how to use it. Google Maps provides a great example of what a truly interactive web page can do, so it was a very fitting choice for my exploration of modern web programming. By combining traditional HTML and JavaScript with Dynamic HTML (DHTML) and Asynchronous JavaScript and XML (AJAX), it is possible to create web pages that act more like traditional applications than static

web pages. I thought that it might make a good senior project. After getting the concept approved by Dr. Briner, I decided to continue my work with the API for my senior project.

Method of Investigation

The method of investigation used for this project most closely fits the evolutionary prototyping model. As mentioned earlier, since this project was designed to provide a learning experience as well as a final product, it was not feasible to fit its design to a typical software development process. A key aspect of evolutionary prototyping is that it “allows the development team to add features, or make changes that couldn't be conceived during the requirements and design phase.” [2] It allows numerous benefits that can be found in my project, such as “the ability to uncover missing or previously unknown requirements” and “the flexibility to meet changing constraints for software systems.” [3] With a high level of flexibility, I was able to focus on the improvement of my project through exploration into different technologies, rather than being forced to continue following an unsound development plan constructed before better knowledge of the subject material was obtained.

While the evolutionary model does allow for a host of benefits, there are also some negative aspects of this model as well. One of the most significant problem, as it applies to this project in particular, is that the evolutionary prototyping model “is particularly susceptible to significant changes in requirements,” [3] which is often referred to as “requirements creep.” As a result of this model's inherent flexibility, it is quite easy to lose focus on the original goal of the project and began to increase the scope of the project well beyond its initial purpose. In my project, this is shown in several areas, including what technologies were used, what functionality was employed, and how the various program structures were developed. This type of expansion is demonstrated in my project, in large part because I was unaware of which technologies would be required to accomplish the goals that were assigned. Several examples of this situation are

discussed in the next section.

Problems Encountered

During the course of this project, I encountered many different problems. Most of the problems just needed extra time to be worked through; others were much larger and took a great deal of experimentation and research to resolve. For a more detailed description of many of the problems I encountered, please refer to my work log. The following problems are those that I consider to be more substantial than most.

JavaScript and the Google Maps API

The first and most important problem I encountered was to learn how to implement the Google Maps API. While a great deal of documentation is available with numerous working examples, I still had a great deal of work to do, especially since I was not familiar with JavaScript. I was directed to one of the best online tutorial websites for web development, w3schools.com [4], which greatly aided my understanding of how to work with JavaScript, HTML, DHTML, AJAX, and numerous other topics. Although w3schools.com was one of the most helpful websites I found, during the course of my project, I have made use of hundreds of websites to obtain guidance on nearly every topic in my project. This was generally accomplished by performing a series of Google searches and sifting through the results until some useful information was found. This process was very quick in some cases, but in other cases it took a great deal of time to find information on some of the more esoteric topics.

By not taking any formal classes on the majority of the technologies my project uses, I was forced to teach myself about many different subjects, with mixed results. The benefits of this method of learning are that the most essential information is learned quickly and it is very easy to see how the different topics are meant to be applied almost immediately. The downside of this approach is that some useful or otherwise important information can be missed, which

creates difficulties that would not exist if the information was known in advance. Overall, I believe the methods I used for this project produced good results, but that more knowledge in the subject area could have allowed for more optimization.

Proper data storage in a cookie

Another major hurdle that I had to overcome for this project was how to store and retrieve various types of information from a cookie. In JavaScript, writing a string of information to the browser's cookie is very simple. As is shown in the code, the cookie is accessed by referring to the `document.cookie` object, and data is written directly to it using the assignment operator (`=`). However, when storing various types of data that must be accessed directly, this can become quite a hassle. To accomplish this, some sort of structure had to be developed to hold the information and have it properly associated with markers and polylines.

After considering different solutions, I finally decided on using a parentheses delimited structure for my marker info. As more data was associated with a marker, the structure expanded until it reached its current configuration which is as follows:

```
(unique marker ID)(marker latitude)(marker longitude)(marker  
comment)(marker color)
```

This structure allows as many markers as the user wants to be stored in the cookie, and the unique ID allows individual cookies to be singled out for alteration or deletion.

While this structure is sufficient to hold marker information, without any other information associated with it, there could only be one person or project's markers stored on a page at a given time. To allow for multiple sessions, I had to set a unique session name for each user, referred to in the code as a `c_name`, which stands for 'cookie name.' By setting a unique session name to a particular set up markers, multiple sessions could be stored in the cookie and loaded when the user changed the session name. This also brought up the need for a structure to

hold the last used session name. To do this, I set a session name with a value that was unlikely to be inadvertently chosen by a user to hold the more recently used session name. The session name I decided to use is `Current_C_name_that_is_being_used_by_this_program`. With this structure in place, there can be multiple sessions and multiple markers.

When setting up unique sessions, it is reasonable that a user would want to have a default location and zoom level to display a certain area of interest, but with only the structures listed above, the user is not capable of holding any information unique to a particular session. This problem was solved by attaching a prefix to the marker information section that held this unique information. The unique session information was separated from the general marker information by a unique delimiter. Initially, I used 'null' as a delimiter, but I discovered that using 'null' could cause errors in JavaScript because 'null' is a reserved word. The current version of the project simply uses DELIMITER as the delimiter between the two sections. The last structure in the cookie, the polyline storage structure, will be discussed later.

Handling multiple click commands

One of the most significant problems that I encountered was how to handle different types of click commands. Within the Google Maps API documentation, there are a great many things that can be done with the click command. Without much effort, a left mouse click can be set to create a marker, center the map, trigger a popup window, zoom in for a closer view of the map or any of the other helpful tasks that are defined in the API. However, to allow the left click do all of these things in the same project requires considerably more effort.

Initially, I was unsure if this was possible until I discovered how to work with the Google Maps Event Listeners. Essentially, these perform much like an asynchronous function calls that are used in AJAX. These listeners are set to do a certain function when an event occurs. However, if set improperly, the code inside the function will be hardwired to do a certain task,

i.e. for the following statement:

```
if(x == 1) do something; else if (x==2) do something else;
```

If x equals 1 at the time the listener is set, then setting x equal to 2 will not have the desired effect. To fix this problem, the listener must be pointed to an outside function so that tests can be performed to accomplish the appropriate task.

Once I discovered this, setting up multiple tasks for a click listener function to perform was much easier. I set up check boxes that correspond to the choices the user can make, and then I have the click listener point to a function, appropriately called `clickListenerFunction`, that acts like a switchboard to determine what will happen when a click event occurs. By dynamically enabling and disabling checkboxes based on user input, I am able to provide a user friendly method of controlling the click listeners without confusion about what action will occur when the user clicks in a given location.

Working with Polylines

The Google Maps documentation explains that “the `GPolyline` object within a Google map denotes a line as a series of points.” These polylines can be assigned a thickness, a color, and a level of opacity. However, these lines, unlike the markers used in the project, are not click sensitive. This means that once the polyline is created, a single segment of the line cannot be selected and modified. This posed a significant problem, since the goal of this project is to allow as much flexibility as possible. To correct this situation, I devised a method of creating polylines where the markers would function as vertexes between lines. This way, if a marker with a line attached was deleted, the line would also be deleted. This allowed for specific portions of a polyline to be removed without removed other polylines from the map.

While this solved a rather significant problem, these polylines would still need to be stored in the cookies along with the rest of the information. Luckily, this turned out to be much

simpler than I originally supposed. After considering the storage of the latitude and longitude of each vertex for a given polyline, I realized that this was unnecessary. Since each marker already had a unique ID associated with it, by simply creating a structure to store two unique IDs, I would have all the information I needed to define a line. The structure I decided upon is as follows:

```
{uniqueMarkerID1, uniqueMarkerID2}
```

I chose to use curly brackets for this application so that the polyline definitions could occupy the same space as the marker information. Since I search through the marker information looking for a leading parenthesis, all the polyline definitions are skipped. The opposite is true when searching for polylines; the marker information is then skipped over. With this structure in place, the cookie can now support all of the data types that are required for my current project.

Database type

Another major consideration for the project was how to expand upon its storage capabilities. Although cookie storage provides the majority of the functionality of a database, it has several distinct disadvantages. One of the most significant disadvantages is that any time the cookie file or folder is removed, all of the data stored for the maps are deleted. Also, many browsers are set, by default, to limit the acceptance of cookies in order to prevent malicious software from harming the computer, which severely limits my projects capabilities. Another significant disadvantage is that it is very difficult to move the stored data from one computer to another, or to backup one's work. So, with these considerations in mind, it was determined a database would provide for even greater functionality of the project.

Deciding on the type of database to be used, however, was a difficult decision. Since JavaScript does not support normal file operations, a backup file could not be written. Also, JavaScript does not support TCP connections, which would allow for data streaming from the

program to the server. A great deal of my time was spent looking into the development of an XML server, but this turned out to be more difficult than I first believed as well. After researching the topic for a while, I decided the best approach would be to set up HTTP connections from my JavaScript code to connect to a server that supported PHP scripts which, in turn, would communicate with a MySQL database. While this solution is more than capable of handling my project's requirements, it brought many new problems.

Working with HTTP connections, AJAX, Apache, PHP and MySQL

The concept for my database setup is fairly straightforward, but, unfortunately, proved to be more difficult than I anticipated. The basic order of connections works as follows:

- 1) The JavaScript code in the web page uses AJAX to send an HTTP request with any appropriate information to a PHP script running on an Apache HTTP server.
- 2) The Apache server, configured for both PHP and MySQL, handles the HTTP request sent by the JavaScript in order to run the PHP script.
- 3) The PHP script runs a SQL query to alter or retrieve information from the MySQL database, then returns the appropriate information back to the web page.
- 4) The web page's JavaScript performs any necessary operations on the returned data and continues running the program.

However, I have encountered many problems with this setup, which is why it is currently non-operational. The first major problem I had was learning how to properly set up my AJAX HTTP request object. I found it to be very difficult to locate good information to explain how to do anything similar to what I am attempting to do. After searching for some information for a couple of weeks, I finally found two reasonably helpful examples, one that used a GET request and one that used a POST request. However, I still had to do a lot of reading before I would have a correct understanding of how an AJAX HTTP request functions.

Many of the other problems I encountered have a lot to do with my inexperience with these technologies. It took some time to learn the basics of how the PHP language works, then some more time to understand how it connects to a MySQL database. Luckily, I had taken Database Management with Mr. Sullivan, so I had some experience with the SQL query language, but working with the data returned from the queries in PHP proved to offer its own challenges.

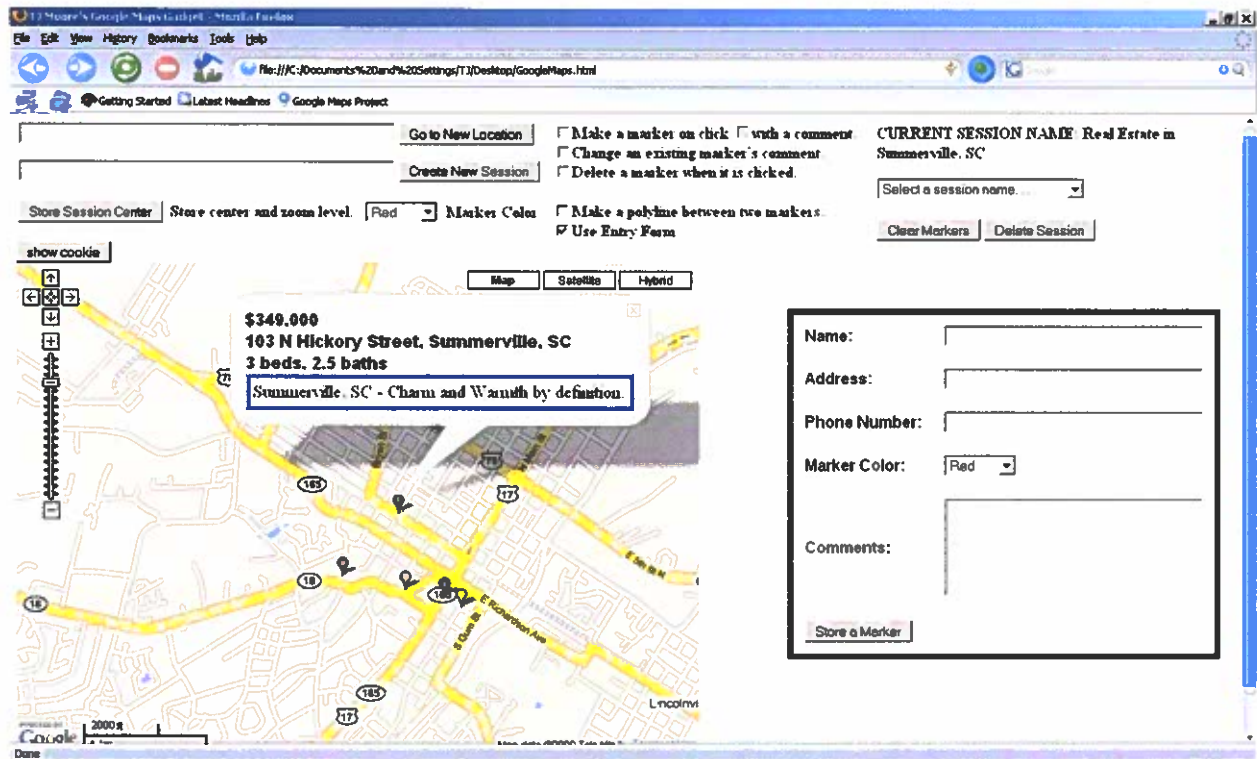
One of the most significant challenges I have faced, and, I believe the reason the database version of my project is still not functioning correctly, is the way the HTTP Request objects work to create connections. As mentioned before, the 'A' in AJAX stands for asynchronous, which means that the requests that are being processed can compete with each other for processing time. Since my project employs many different HTTP request for its different functions, unique HTTP request objects must be created so that they are not able to overwrite each others status information while they are running. However, even with the unique ids, I believe there may be a problem with running so many of these objects. At the moment, I do not have a clear idea of what the problem is, and more time is needed to determine how best to handle these issues.

Application

As I mentioned before, there are many creative uses of this technology all over the internet. Two of my favorite uses of the Google Maps API are found at stormadvisory.org [5], which has a hurricane tracker, and trulia.com [6], which shows real estate information. Both of these sites provide excellent information and show what can be done with Google Maps. Unlike those websites, my project allows an end user to create unique, personalized displays of anything that may be of interest to him or her. So, since this project is meant to be a generalized tool, I will include a few examples of what it can do.

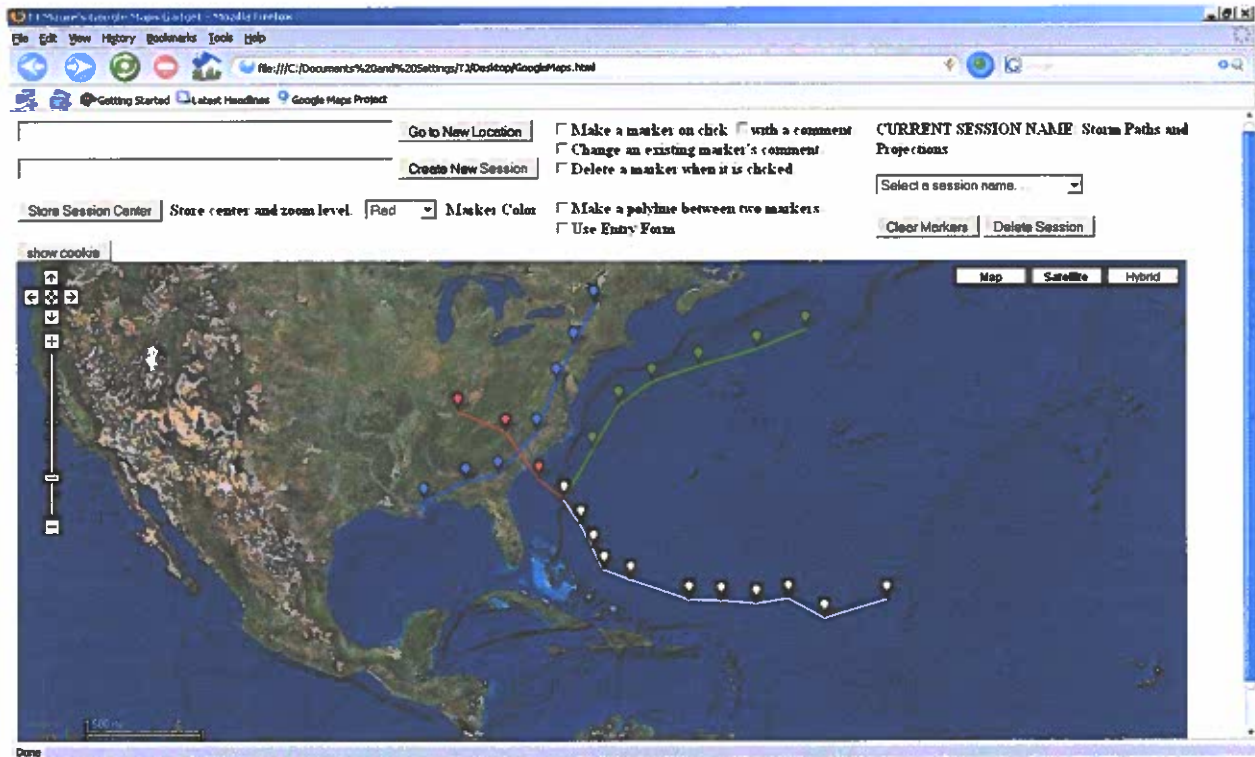
Real Estate for Sale in a Given Area

This use of my project shows location of current real estate. Inside of the popup information window is the price of the house, the address, the number of bedrooms and bathrooms and whatever comments may be helpful for the potential purchaser.



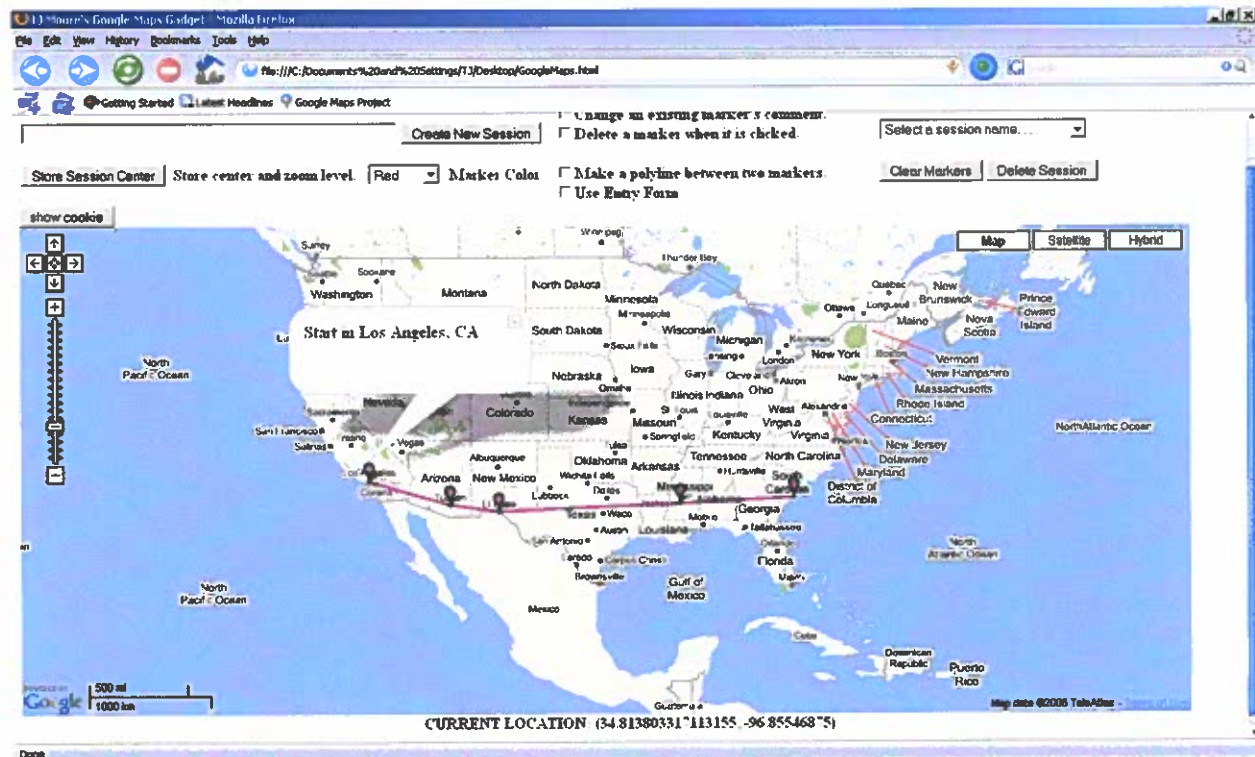
Storm Paths and Projections

The next screen shot shows the path of a fake hurricane. The popup windows give the date of the position. The white path is where the hurricane has been. Two potential paths are shown; a more likely path in green and a less likely path in red. The blue polyline is meant to simulate a low pressure system.



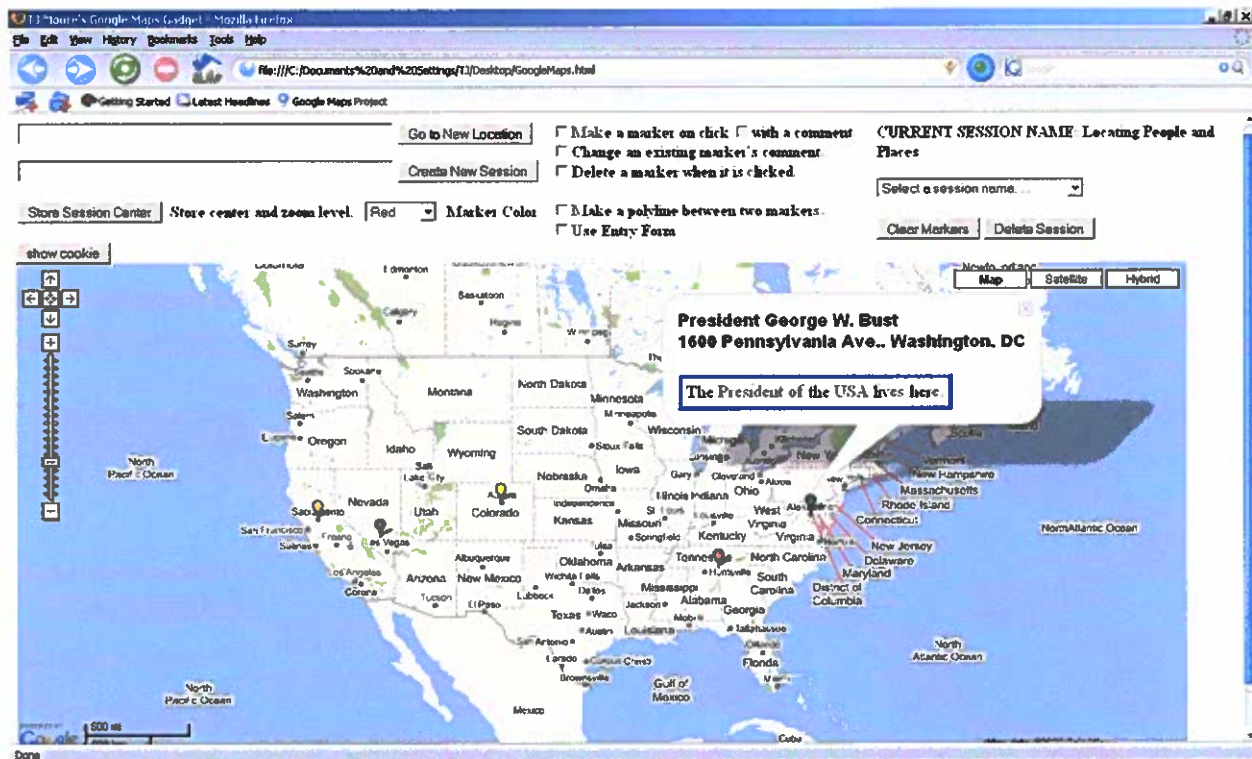
Tracking Shipments

This shows the path of a package being shipped from Los Angeles, CA to Charleston, SC.



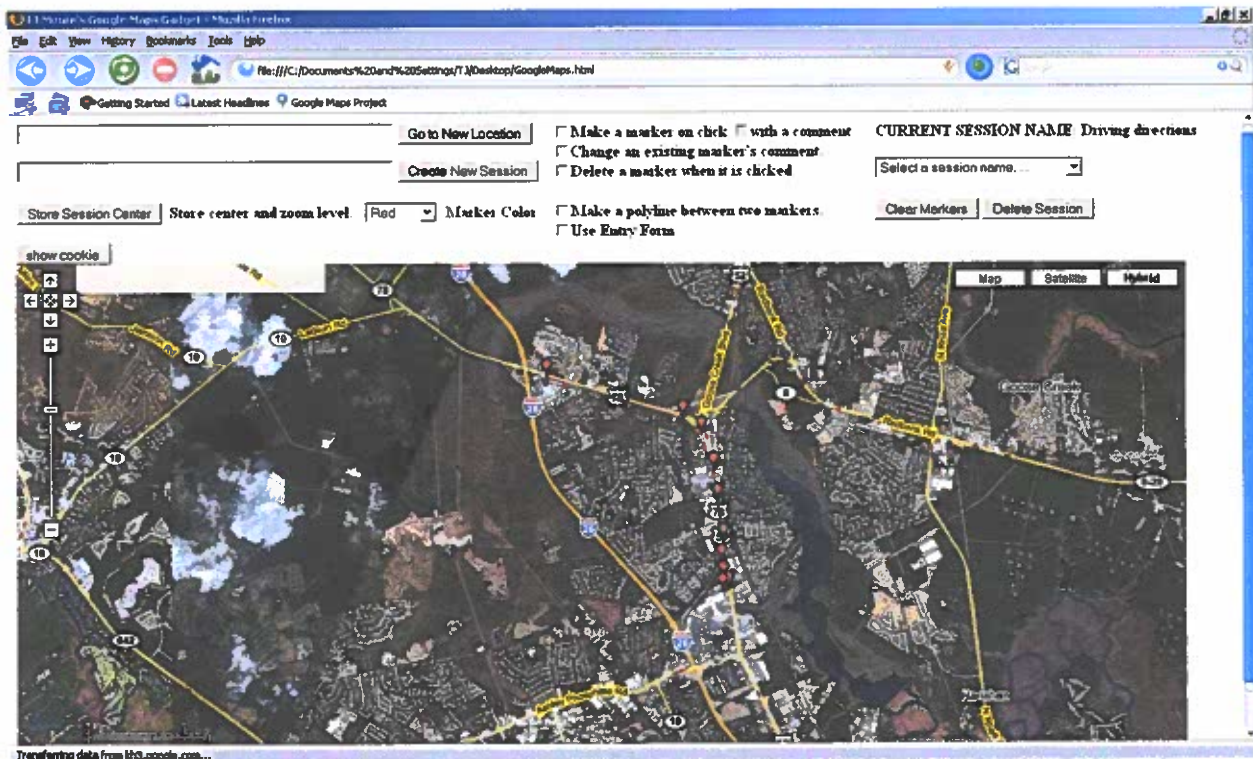
Locating People and Places

This shows some random people and places of interest. The information in the popup windows was formatted using the entry form to allow for additional structuring of the information.



Driving Directions

The next screen shot gives directions from Charleston Southern University to Northwood's Mall. Inside the popup windows, there are helpful comments and directions concerning when to take a turn or what lane the driver should be in at what point.



Future Work

With any tool, there are almost always improvements that can be made. I would like to see the following items in my project accomplished.

Fix version 2

Since my database version of the project is currently nonfunctional, I would like to see it fixed. I believe if I had more time to research and debug the code, it would be able to run correctly. There is a great deal of potential for the database version that is above and beyond the capabilities of the cookie based version that I would like to see made available to the end users. Once finished, data could be easily shared among various users and allow for more permanent storage of a persons work. I would hope this would promote greater use of the tool and allow new uses for it.

Add comment data validation in version 1

With the benefits of being able to add unique user-given data into the popup windows,

there is also a great deal of opportunity to cause problems with the information parsing used by the program. For example, since parentheses are used to delimit marker information, added them into marker comments can cause errors when the program begins reading the data that is stored in the cookie. By adding some data entry controls, this problem could be largely eliminated, but it will take some time to determine exactly what can or cannot be allowed in the cookie.

Streamline for use with Internet Explorer

As of right now, Internet Explorer is supported, but it does not run as well or look as clean as it does in Mozilla Firefox. To make the tool widely accessible, it would need to offer better support for users of Internet Explorer. The user interface clean up would most likely be fairly simple, but the optimization that is also required might take considerably more time. The slowness of the projects operation in IE may be contributed to small syntax errors that Firefox ignores, or it may reflect a significant difference in how the two browsers process information. While comparing Internet Explorer 7 with Mozilla Firefox 2.0 when using maps.google.com [7], their run speed is comparable, so the map itself is most likely not the reason for IE's slowness when running the project. More debugging is necessary to determine the cause of the problem.

Add a mechanism to transfer cookies

To offer greater user support for the cookie based version, a mechanism for storing a user's cookies to a file and copying that information back to their cookie file or folder would be extremely valuable. Since JavaScript does not offer file operations, some other method would need to be employed to allow for this capability. However, if this could be developed, I believe it could be made to offer some of the benefits of the database based version without the need to create the database itself.

Allow customization of entry form

My final idea for how this project could be improved would be the addition of

customizable entry forms. The current cookie version has a basic entry form that can be used to store a good bit of information and format it using cascading style sheets to offer some formatting within the markers' popup windows. In the future, it would be beneficial to allow the end user to have some level of customization of what the form can store and how it formats the results. This could also lead to greater flexibility that may foster increased use of the tool.

Conclusion

This project may show some of the potential of this type of mapping, but many additional capabilities are certainly possible. The longer this technology is readily available, the more creative programmers will develop new and exciting applications with it. Hopefully, this project will spark some greater interest into the potential of web mapping and allow people to begin thinking what else can be done to make these tools more useful in their daily lives. Overall, there is still a lot of work that can be accomplished and many capabilities that have yet to be imagined.

Web References

- 1) The Google Maps API: <http://www.google.com/apis/maps/>
- 2) Explanation of Software Prototyping: http://en.wikipedia.org/wiki/Software_prototyping
- 3) Research concerning Evolutionary Prototyping:
<http://www.csc.ncsu.edu/faculty/anton/pubs/epamRE01.pdf>
- 4) Web development resources <http://www.w3schools.com>
- 5) Storm tracking with Google Maps: <http://stormadvisory.org/map/atlantic/>
- 6) Real Estate mapping with Google Maps: <http://www.trulia.com/>
- 7) Google's online mapping site: <http://maps.google.com>

Google Maps Gadget: An Application of the Google Maps Toolkit

TJ Moore
December 13, 2006

Introduction

Project Purpose

The purpose of this project is to do two things:

To give non technical users the ability to work with many of the powerful features presented in the Google Maps API

To allow me the opportunity to become familiar with various network programming tools and procedures.

Project Versions

There are two main versions:

Version 1 employs cookie-based storage

Version 2 uses a MySQL database to store the data

Version 1: Cookie-Based Data Storage

- This cookie-based version is designed to be a "light client" that requires no setup
- A cookie is a small text file that holds information for a browser
- The entire project is very small, just over 43 KB, and requires only an internet connection and a browser.

Version 2: Server-Based Data Storage

- The database version is considerably more complicated
- Data is stored on an Apache HTTP server, using PHP scripts and a MySQL database.
- The particular package used for this project is VertrigoServ

found at <http://vertrigo.sourceforge.net/>

Version 2: Server-Based Data Storage (continued)

- Version 2 uses AJAX requests sent from the main JavaScript file that work with PHP scripts running on the server
- These scripts perform the necessary SQL queries to create, read, update or delete the data

Usage Information

- For both project versions, Mozilla Firefox is the recommended browser.



- For Version 1, please ensure that long term cookie storage is enabled

General User Interface



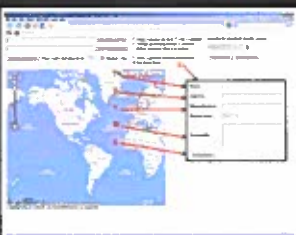
- A Store Session Center Button
- B Marker Color Drop Down Menu
- C Create New Session Button
- D Go to New Location Button
- E Entry Form Check-box
- F Polyline Creation Check-box
- G Delete Marker on Click Check-box
- H Change Marker Comment Check-box
- I Make a Marker on Click Check-box
- J Add a Comment Check-box

General User Interface



- K Current Session Name Display
- L Session Drop Down Menu
- M Clear All Markers Button
- N Delete Session Button
- O Pan and Zoom Control
- P Map Type Toggle Buttons
- Q Map Scale
- R Current Location Display

Popup Form User Interface



- S Entry Form
- T Name Text Box
- U Address Text Box
- V Phone Number Text Box
- W Marker Color Drop Down Menu
- X Store a Marker Button

Usability Examples

Method of Study

Evolutionary Prototyping

- My method of study most closely fits the **Evolutionary Prototyping Model**
- Some of the main concepts behind this model are:
 - To build a very robust prototype in a structured manner and constantly refine it
 - To allow the system to be continually refined and rebuilt

Evolutionary Prototyping (continued)

- To allow the developer to add features or make changes that couldn't be conceived during the requirements and design phase
- A product is never "done" it is always maturing as the usage environment changes

Source: <http://encl.com/2006/06/26/evolutionary-prototyping/>

Evolutionary Prototyping (continued)

- Problems with this model
 - This model is particularly susceptible to significant changes in requirements, often referred to as "requirements creep."
 - As a result of this model's inherent flexibility, it is quite easy to lose focus on the original goal of the project

Problems Encountered

Problems

- JavaScript and the Google Maps API*
 - Learning the Google Maps API and Javascript
- Proper data storage in a cookie*
 - Learning how to store and retrieve various types of information from a cookie
 - Setting up my own data structure inside of the browser's cookie

Problems

- *Handling multiple click commands*
 - Discovering how to work with the Google Maps Event Listeners so multiple actions can be performed with a click.
- *Working with Polylines*
 - Set up modifiable polylines
 - Add some type of click response ability

Problems

- *Database type*
 - Determine the best type of database to use
 - Working past XML and TCP connections
- *Working with HTTP connections, AJAX, Apache, PHP and MySQL*
 - Setting up connections to allow for interoperability between technologies

Future Work

Future Work

- *Fix version 2*
 - Version 2 needs to be debugged and may require a new technique to become functional
- *Add comment data validation in version 1*
 - Validate any information to prevent errors
- *Streamline for use with Internet Explorer*
 - Since Internet Explorer is used so extensively, the project should work more efficiently

Future Work

- *Add a mechanism to transfer cookies*
 - To expand the functionality of Version 1, a mechanism should be developed to save and store the cookie information
- *Allow customization of entry form*
 - The entry form used in Version 1 should be made customizable and, eventually, added to Version 2 as well

Table of Contents

Introduction	3
Background	3
Project Purpose	3
Project Versions	3
User Interface Description	4
Version 1: Cookie-Based Data Storage	7
Premise	7
Function List	7
Group I: Initialization	7
Group II: Creation	8
Group III: Revision	8
Group IV: Deletion	9
Group V: Miscellaneous	9
Version 2: Server-Based Data Storage	10
Premise	10
Function List	10
<i>JavaScript Function List</i>	<i>10</i>
Group I: Initialization	10
Group II: Creation	11
Group III: Revision	11
Group IV: Deletion	12
Group V: Miscellaneous	12
<i>PHP Function List</i>	<i>12</i>
Group I: Initialization	12
Group II: Creation	13
Group III: Revision	13
Group IV: Deletion	13
Group V: Miscellaneous	13
Conclusion	13
Appendices	14
Appendix A: Version 1 Code	14
Appendix B: Version 2 Code	42
Appendix C: PHP Functions	71
Appendix D: Project Log	83

Introduction

Project Background

Although computerized aided mapping has been in use since the mid-1900s, the first operational geographic information system (GIS) was developed in 1964 by the Canadian Department of Energy, Mines, and Resources. It was called the "Canadian Geographic Information Systems" (CGIS) and was in use into the 1990s. Because of the success of the CGIS, many commercial companies were also created to provide GIS capabilities for a fee (http://en.wikipedia.org/wiki/Geographic_information_system).

Recently, web based mapping systems have been increasingly available to the public for free use. Services such as MapQuest, Yahoo Maps, and Google Maps provide various degrees of functionality to their users. To allow web developers to have even greater control over there mapping systems, several of these services have created application programming interfaces (APIs). The first version of the Google Maps API was released on February 8th, 2005, and version two was released on April 3rd, 2006, which added more functionality. The Google Maps added geocoding on June 11th, 2006, which allows for address information to be interpreted and latitude and longitude values to be returned (http://en.wikipedia.org/wiki/Google_Maps). While Google's API toolkit may be less extensive then many of the proprietary GIS systems, it provides a great deal of functionality at no cost to its users.

With the help of these new services, web developers have the ability to use mapping data in ways that would have been much less feasible just a few short years ago. With that in mind, the goal of this project is to make this level of functionality available to non-technical users.

Project Purpose

The purpose of this project is to give non-technical users the ability to work with many of the powerful features presented in the Google Maps API, as well as to allow me the opportunity to become familiar with various network programming tools and procedures.

Project Versions

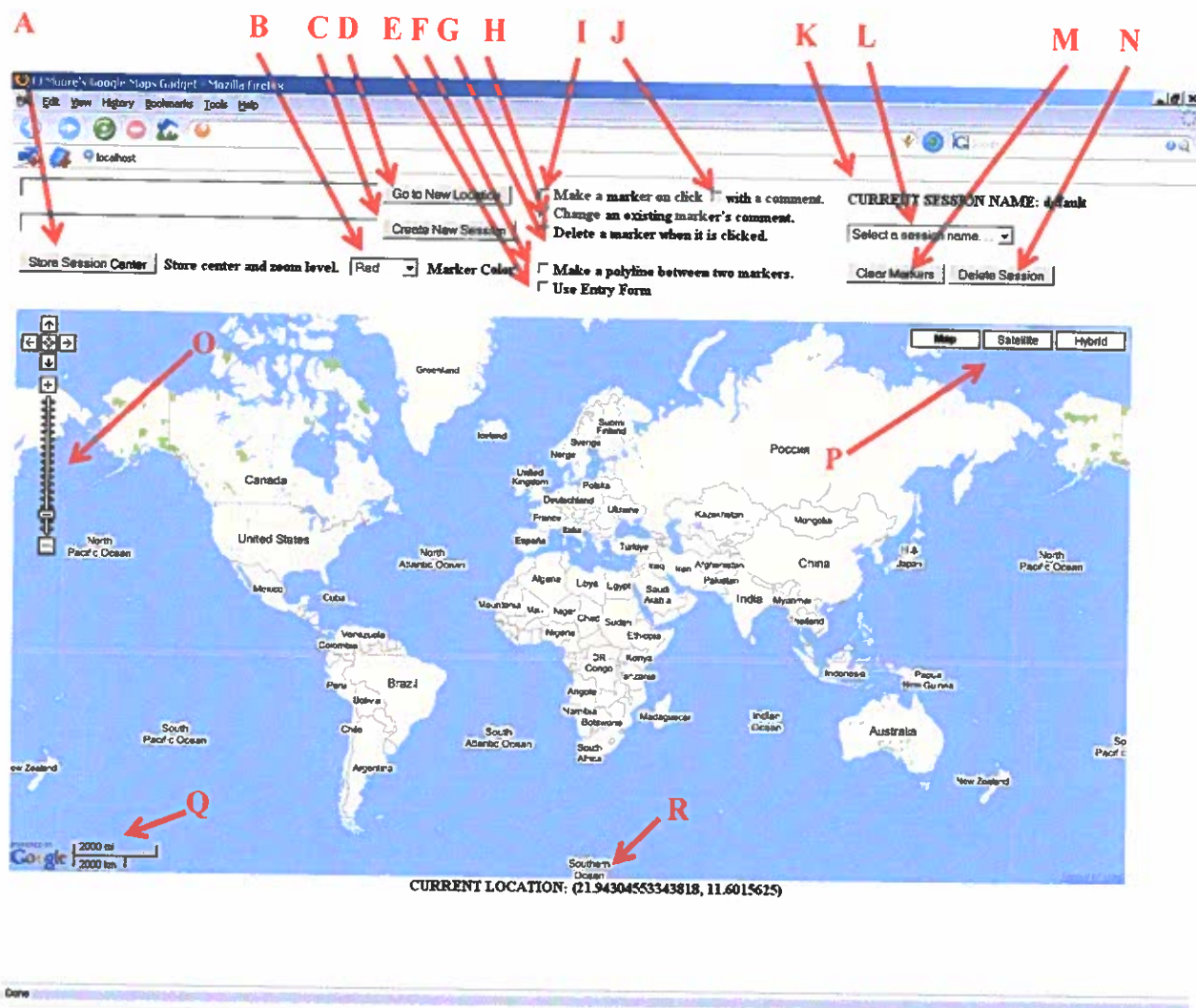
This project consists of two main versions; one which employs cookie-based storage and another that uses a MySQL database to store the data. The cookie-based version is designed to be a light client that requires very little setup to deploy on a local machine. The entire project is just over 43 KB, and only requires an internet connection and a browser. Since all stored data is kept in the browsers cookies, there is no need to connect to any additional servers besides those maintained by Google.

The database version of the project's storage mechanism is considerably more complicated, and, unfortunately, at this time is still being debugged. Its data is stored on an Apache HTTP server that is setup to use PHP scripts and a MySQL database. The particular package used for this project is VertrigoServ, which can be found at <http://vertrigo.sourceforge.net>. This version uses a series of AJAX requests sent from the main JavaScript file that connect to PHP scripts running

on the server, which, in turn, perform the necessary SQL queries to create, read, update or delete the data.

For both project versions, Mozilla Firefox is the recommended browser. For Version 1, please ensure that long term cookie storage is enabled.

User Interface Description



A. Store Session Center Button

This button allows the user to store the current location as the starting location for the current session.

B. Marker Color Drop Down Menu

This drop down menu changes the marker color currently being used. It allows the user to make markers in ten different colors: red, orange, brown, yellow, green, blue, purple, black, gray and white.

C. Create New Session Button

This button creates a new session for the user with the name given in the associated text box.

D. Go to New Location Button

This button uses Google's geocoder to jump to the address given in the associated text box.

E. Entry Form Checkbox

This checkbox displays the entry form (please see next image for description).

F. Polyline Creation Checkbox

This checkbox allows the user to make a polyline between two points on the map, two existing markers, or one map point and one marker. The color of the line will match that of the first marker used to define the polyline.

G. Delete Marker on Click Checkbox

This checkbox allows the user to delete individual markers by clicking on them.

H. Change Marker Comment Checkbox

This checkbox allows the user to change the information in an existing marker's popup window.

I. Make a Marker on Click Checkbox

This checkbox allows the user to create a new marker by clicking on the appropriate location.

J. Add a Comment Checkbox

Used in conjunction with the 'Make a Marker on Click Checkbox,' this checkbox allows the user to put a comment in the marker's popup window as soon as it is created.

K. Current Session Name Display

This displays the name of the session currently being used.

L. Session Drop Down Menu

This drop down menu contains all of the sessions currently available and allows the user to switch between sessions.

M. Clear All Markers Button

This button clears all of the markers from the current session.

N. Delete Session Button

This button permanently deletes the current session.

O. Pan and Zoom Control

These controls allow the user to change the location and zoom level of the map.

P. Map Type Toggle Buttons

These buttons change the type of map being displayed.

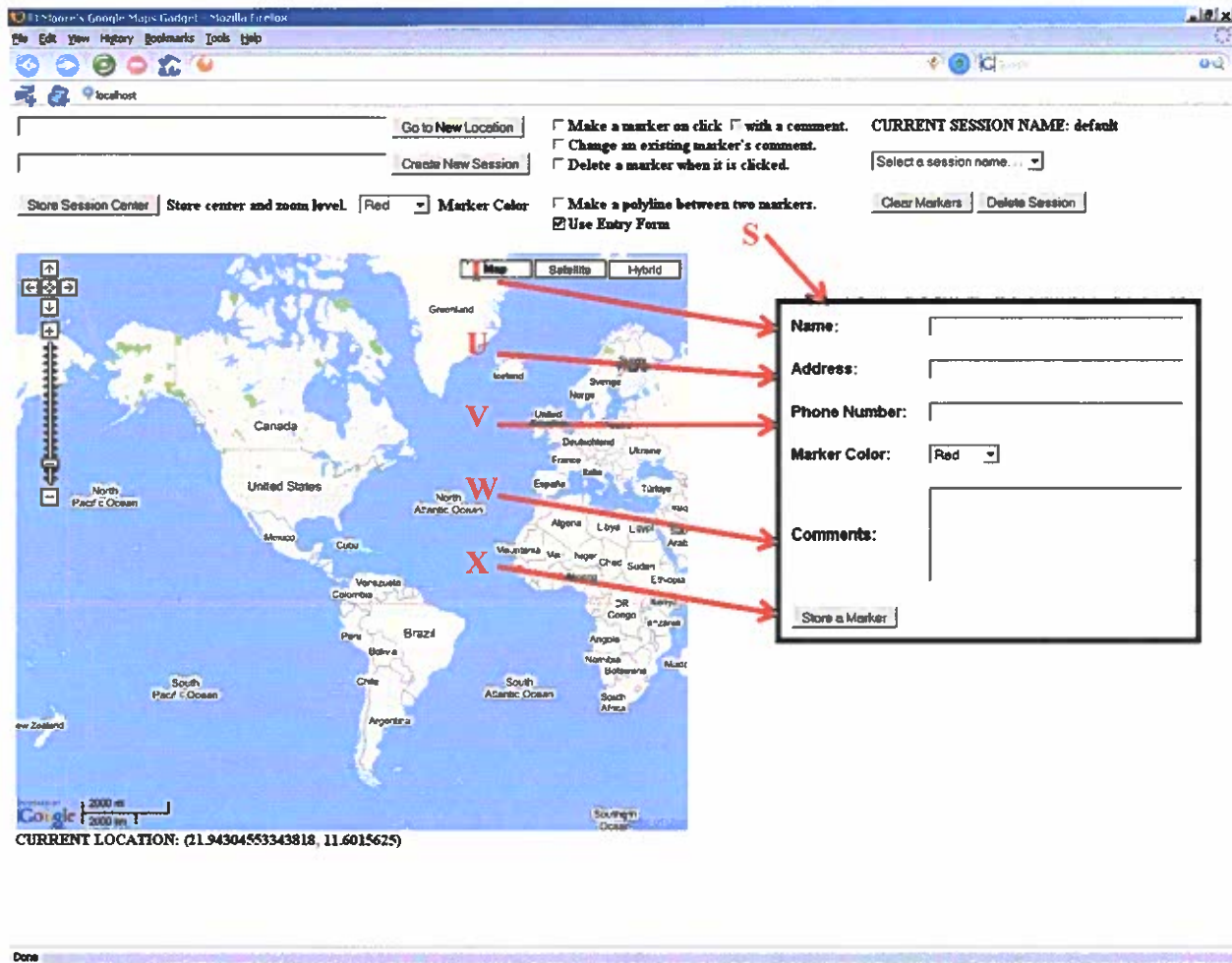
Q. Map Scale

This shows the current map scale.

R. Current Location Display

This gives the current latitude/longitude location of the center of the map.

Please Note: The following entry form description only applies to version 1.



S. Entry Form

This form allows the user to enter a person or organizations name, address and phone number, as well as any necessary comments, into the popup window of a marker. The address field works with Google's geocoder to find the address given and make a marker at that point. The information will be formatted before it is stored. Please note: if the address cannot be found, the marker will not be created.

T. Name Text Box

This text box holds a name.

U. Address Text Box

This text box holds an address and operates as described above.

V. Phone Number Text Box

This gives text box holds the phone number given.

W. Marker Color Drop Down Menu

This drop down menu works like the main color drop down menu to provide color options for the markers being created.

X. Store a Marker Button

This gives button stores the information given in the markers popup window.

Version 1: Cookie-Based Data Storage

Premise

The main premise of this project is to allow non-technical users to make as much use of modern open source mapping technology as possible. By designing a mostly point-and-click interface that requires no knowledge of the underlying code, the project attempts to ensure a quick and easy method to show many different types of geospatial information. Tracking shipments, locating people and places, showing storm paths and projections and giving driving directions are just a few of the functions that this project is designed to perform. The structure of the data being stored in the cookie is as follows:

```
aSessionName[sessionLatitude][sessionLongitude][sessionZoomLevel]DELIMITER(markerUniqueID)(markerLatitude)(markerLongitude)(markerComment)(markerColor)(markerUniqueID)(markerLatitude)(markerLongitude)(markerComment)(markerColor){uniqueMarkerID1, uniqueMarkerID2}
```

```
anotherSessionName[sessionLatitude][sessionLongitude][sessionZoomLevel]DELIMITER(markerUniqueID)(markerLatitude)(markerLongitude)(markerComment)(markerColor)(markerUniqueID)(markerLatitude)(markerLongitude)(markerComment)(markerColor){uniqueMarkerID1, uniqueMarkerID2}
```

```
Current_C_name_that_is_being_used_by_this_program(aSessionName)
```

The spacing has been changed and some miscellaneous data has been removed, but the general structure has been left intact. This shows two sessions, each with two markers and a polyline, followed by the current session name holder. The session name `Current_C_name_that_is_being_used_by_this_program` was used so that there would be little chance of it being picked as an actual session name.

Function List

Group I: Initialization

init()

Initializes items and sets up portions of the program that only need to be called once. It calls the `setSessionName()` and `load()` functions.

load()

Loads controls, holds the lat/long display listener, and calls the cookie functions.

setSessionName()

Sets the current session name being used by the program.

getSessionCenter()

Gets the current center and zoom level for the current session.

loadSessionCenter()

Loads the center and zoom level from the cookie.

getSessionNames()

Gets the session names from the cookie and displays them in the session drop down menu.

useCookies()

Breaks down the cookie into elements that can be used to create markers. It calls `parseString()`, `createMarker()` and `getPolyLines()`.

parseString()

Goes through the middle portion of cookie returned from `getCookie()` and breaks it into individual data elements which are returned to `useCookies()`.

getPolyLines()

Gets the polyline data from the cookie. It calls `parsePolyLines` and `makeNewPoly()`.

parsePolyLines(lastIndex)

Parses the polyline information and returns it to `getPolyLines()`.

Group II: Creation**storeNewSession(tempCname)**

Stores new session name in a cookie. It calls `changeSessionName()`.

checkCookieForSession(tempCname)

Checks to see if the session name is already in the cookie so that the name will not be overwritten. Returns location of session name or -1 if not found.

storeSessionCenter()

Stores the current center and zoom level in the cookie. Calls `getCookie()`, `getSessionCenter()` and `setCookie()`.

updateCookie(newInfo)

Stores any new information given in the cookie. Calls `getCookie()` and `setCookie()`.

createRecord(location, message)

Creates a record in the cookie, then calls `createMarker()` to create the marker. It returns a marker object to the calling function.

createMarker(point, message, info)

Creates markers with lat/long, info window comment and a unique identifier. It returns a marker object to the calling function.

makeNewPoly(markerID, newMarker)

Makes a new polyline between two markers. It calls `getMarkerInfo()` and `storePoly()`.

storePoly(ID1, ID2)

Stores the polyline data in the cookie. It calls `getCookie()` and `updateCookie()`.

Group III: Revision**changeSessionName(tempCname)**

Changes the current session name that is being used by the program.

changeComments(markerID)

Changes the comments of existing markers. It calls `getMarkerInfo()`, `deleteOneMarker()`, `updateCookie()` and `createMarker` to delete the marker, then recreate it using all of the same information except for the comment.

iconColor(changedColor)

Changes the color of the icons used by the program. It also ensures both color drop down menus are set to the same color.

Group IV: Deletion**deleteSession()**

Deletes all session information from the cookie.

deleteOneMarker(info, deletePoly)

Deletes the information of the particular marker. It uses `getCookie()` and `setCookie` to delete the marker. It also uses `deletePolylines()` to delete any polylines associated with the marker.

deletePolylines(info, startLocation)

Deletes all polylines that contain the `markerID` of the marker being deleted. It uses `getCookie()`, `setCookie()` and `searchBackwards()`.

searchBackwards(index, searchItem)

Used when the ID being looked for is the second element in the polyline definition to go back and get the location of the first element.

deleteMarkers()

This clears all markers from the map and removes the data from the cookie. It uses `getCookie()`.

Group V: Miscellaneous**getCookie()**

Gets the contents of the cookie and returns the usable portion to the calling function.

setCookie(info)

Stores the updated contents of the cookie.

countCookies()

Counts the number of elements in the cookie and stores the values in global variables.

clickListenerFunction(marker, point)

One of the key functions to this project, this function determines what function a left mouse click will perform based on which checkboxes are or are not checked.

getMarkerInfo(markerID)

Given a marker ID, this function returns the values of a marker in an array to the calling function. It uses `getCookie()` to retrieve the marker data.

markerClickListener(marker, message)

Used to create the marker's default listener, which opens an info window above the marker.

disableComment()

Determines which checkboxes should be disabled and which should be disabled.

showAddress(address)

Sends information entered into the Go to New Location text box to the geocoder listener.

showCookie()

Used only for debugging purposes, displays the contents of the cookie.

Version 2: Server-Based Data Storage

Premise

The premise of this version of the project is similar to that of the cookie based version, but it uses a database to store the session, marker, and polyline information instead of storing that data in the cookie. It also allows non-technical users to make use of modern open source mapping technology and has the same mostly point-and-click interface as the cookie version. The main difference in this version is the ability to store the data remotely and more permanently than the cookie version. This version is currently in the debugging stage, but with more time, it should be functional. It will maintain the capability to track and locate items of interest, but it does not currently employ the entry form that is available for the cookie version. This could be added at a later date after debugging is complete.

Function List

JavaScript Function List:

Note: For all of the functions that require processing of data returned from the MySQL database, the function used to process the data will be named 'finish' plus the name of the function setting up the HTTP request object. These functions will be listed with the setup functions.

Group 1: Initialization

init()

Initializes items and sets up portions of the program that only need to be called once. It calls the `setSessionName()` and `load()` functions.

load()

Loads controls, holds the lat/long display listener, and calls the `loadSessionCenter()` and `parseString()` functions.

makeRequest()

Sets up the HTTP request object based on the type of browser being used. It is set up to handle Internet Explorer and Firefox browsers. It returns an object id to the calling function.

setSessionName() and finishSetSessionName()

Set the current session name being used by the program. The `finishSetSessionName` function calls `storeNewSession()` and `storeSessionCenter()`. `setSessionName()` calls `makeRequest()` and uses a GET request object.

getSessionCenter()

Gets the current center and zoom level for the current session. It calls `changeSessionName()`.

loadSessionCenter() and finishLoadSessionCenter()

Loads the center and zoom level from the database. `loadSessionCenter()` calls `makeRequest()` and uses a GET request object.

getSessionNames() and finishGetSessionNames()

Gets the session names from the database and displays them in the session drop down menu. `getSessionNames()` calls `makeRequest()` and uses a GET request object.

parseString() and finishParseString()

Gets the marker information from the database and creates markers from the data given. finishParseString() calls createMarker() and getPolyLines(). parseString() calls makeRequest() and uses a GET request object.

getPolyLines() and finishGetPolyLines()

Gets the polyline data from the database. finishGetPolyLines() calls makeNewPoly(). getPolyLines() calls makeRequest() and uses a GET request object.

Group II: Creation**storeNewSession(tempCname)**

Stores new session name in the database. It calls makeRequest() and changeSessionName(), and it uses a POST request object.

checkForSession(tempCname) and finishcheckForSession()

Checks to see if the session name is already in the database so that the name will not be overwritten. checkForSession() calls makeRequest() and sets up a GET request object.

storeSessionCenter()

Stores the current center and zoom level in the database. Calls getSessionCenter() and makeRequest(), and it uses a POST request object.

createRecord(location)

Creates a record in the database, then calls createMarker() to create the marker. It also calls makeRequest() and uses a POST request, it then returns a marker object to the calling function.

createMarker(point, message, info)

Creates markers with lat/long, info window comment and a unique identifier. It returns a marker object to the calling function.

makeNewPoly(markerID, newMarker)

Makes a new polyline between two markers. It calls getMarkerInfo() and storePoly().

storePoly(ID1, ID2)

Stores the polyline data in the database. It calls makeRequest() and uses a POST request object.

Group III: Revision**changeSessionName(tempCname)**

Changes the current session name that is being used by the program. It calls makeRequest() and uses a POST request object.

changeComments(markerID)

Changes the comments of existing markers. It calls getMarkerInfo(), deleteOneMarker(), makeRequest() and createMarker to delete the marker, then recreate it using all of the same information except for the comment. It uses a POST request object.

iconColor(changedColor)

Changes the color of the icons used by the program.

Group IV: Deletion**deleteSession()**

Deletes all session information from the database. It calls `makeRequest()` and uses a POST request object.

deleteOneMarker(info, deletePoly)

Deletes the information of the particular marker. It calls `makeRequest()` and uses a POST request object. It also uses `deletePolylines()` to delete any polylines associated with the marker.

deletePolylines(info)

Deletes all polylines that contain the `markerID` of the marker being deleted. It uses `makeRequest()` and uses a POST request object.

deleteMarkers()

This clears all markers from the map and removes the data from the database. It calls `makeRequest()` and uses a POST request object.

Group V: Miscellaneous**clickListenerFunction(marker, point)**

One of the key functions to this project, this function determines what function a left mouse click will perform based on which checkboxes are or are not checked.

getMarkerInfo(markerID) and finishGetMarkerInfo()

Given a marker ID, this function returns the values of a marker in an array to the calling function. `getMarkerInfo()` calls `makeRequest()` and uses a GET request object. `finishGetMarkerInfo()` calls `createMarker()` and `getPolylines()`.

markerClickListener(marker, message)

Used to create the marker's default listener, which opens an info window above the marker.

disableComment()

Determines which checkboxes should be disabled and which should be disabled.

showAddress(address)

Sends information entered into the Go to New Location text box to the geocoder listener.

PHP Function List

Each of these PHP files corresponds to the function of the same name and executes queries to complete the purpose of the function that calls it.

Group 1: Initialization

setSessionName.php

loadSessionCenter.php

getSessionNames.php

parseString.php

getPolyLines.php

Group II: Creation

storeNewSession.php
checkForSession.php
storePoly.php
createRecord.php
storeSessionCenter.php

Group III: Revision

changeSessionName.php
changeComments.php

Group IV: Deletion

deleteSession.php
deleteOneMarker.php
deletePolylines.php
deleteMarkers.php

Group V: Miscellaneous

getMarkerInfo.php

Conclusion

Web based mapping, like those capabilities provided by Google Maps, will undoubtedly continue to increase in both detail and functionality. This project attempts to put as much of the current capabilities into the hands of end users so that they can use it for whatever purposes they see fit. The purpose of this project was to be flexible enough so as not to limit how it could be used.

There is still a great deal of work to be done to improve on this tool. The structure of the code and the methods employed could be optimized for greater efficiency and wider range of use, Version 2 still needs to be debugged and streamlined before it can be deployed as a web service and, as Google creates more modules, new functionality could be added. However, a great deal has been accomplished. The user is able to make multiple sessions, store markers, polylines, and comment information, as well as make use of a customized entry form and various other controls. Overall, the project's goal of making an application-like web mapping tool was largely accomplished, and it will hopefully be useful to anyone who needs to use this type of mapping capability.

Appendices

Appendix A: Version 1 Code

```

<!DOCTYPE html PUBLIC "-//W3C//DTD XHTML 1.0 Strict//EN"
"http://www.w3.org/TR/xhtml1/DTD/xhtml1-strict.dtd">
<html xmlns="http://www.w3.org/1999/xhtml" xmlns:v="urn:schemas-microsoft-com:vml">
<head>
  <meta http-equiv="content-type" content="text/html; charset=utf-8"/>
  <title>TJ Moore's Google Maps Gadget</title>

  <script type="text/javascript">
    if (is_ie5up)
      //this allows polylines to be shown in internet explorer
      document.write("<style type='\"text/css\"'> v\:* {behavior:url(#default#VML);}</style>");
  </script>

  <script src="http://maps.google.com/maps?file=api&v=2&key=ABQIAAAA3jhyBUYFZJISYJPUEKR6HRQkg5m-
Xez6dYoj0OQawlVuWjdFrXRDWvGBOFGEiMa_2i_U7yV_0v1hPw"
    type="text/javascript"></script>

  <script type="text/javascript">

    //<![CDATA[

    var map;
    var geocoder;
    var parseStringStart = 0;
    var parseStringEnd = 0;
    var flag = 1;
    var items = 0;
    var polyItems = 0;
    var c_name;
    var icon;
    var color = "red";

    var centerLat;
    var centerLong;
    var centerZoom;

    var polyline = new Object(); //holds the polylines in object variables
    var currentMarkerID = 0; //holds the unique ID of the current marker

    var string = "";
    //stores what is received from httpRequest

    //general variable for map object
    //used to store geocoder object
    //used to store the start of an individual data object from the cookie
    //used to store the end of an individual data object from the cookie
    //initialized to true, used by parseString() and useCookies()
    //stores number of markers
    //stores number of polylines
    //stores the c_name that corresponds to the user/session name given
    //variable used for custom icon
    //variable used for the icon's current color, default is red

    //variables used to hold location of map center
  ]>

```

```

function init()           //this function initializes items that only need to be called once
{
    if (GBrowserIsCompatible())
    {
        map = new GMap2(document.getElementById("map"));           //map object, first created above

        geocoder = new GClientGeocoder();                           //geocoder object

        document.colorOptions.reset();           //resets the radio buttons
        document.geocode.reset();               //resets the go to location field
        document.session.reset();              //resets the create new session field
        document.makeCheckBox.reset();          //resets the makeCheckBox checkbox
        document.deleteMarkerCheckBox.reset();  //resets the deleteMarker checkbox
        document.makeCheckBox.make.disabled = false;           //enables the makeCheckBox checkbox
        document.deleteMarkerCheckBox.deleteMarker.disabled = false; //enables the deleteMarker checkbox
        document.makeCheckBox.comments.disabled = true;        //resets the commentCheckBox checkbox to disabled
        document.changeCommentCheckBox.reset(); //resets the change comment checkbox
        document.changeCommentCheckBox.disabled = false;        //enables the change comment checkbox
        document.makePolyLine.reset();          //resets the make polyLine checkbox
        document.makePolyLine.makePoly.disabled = false;       //enables the make polyLine checkbox
        document.hideMap.reset();              //resets the use entry form checkbox
        document.getElementById("colorDropDownMenu2").value = "red"; //resets form's drop down menu

        setSessionName();

        //This block initializes the icons to be used in the project
        icon = new GIcon();
        //makes a new icon object
        icon.image = "http://labs.google.com/ridefinder/images/mm_20_red.png"; //gets red icon image
        icon.shadow = "http://labs.google.com/ridefinder/images/mm_20_shadow.png"; //gets icon's shadow
        icon.iconSize = new GSize(12, 20); //sets icon's size
        icon.shadowSize = new GSize(22, 20); //sets shadow's size
        icon.iconAnchor = new GPoint(6, 20); //anchors the icon to the map
        icon.infoWindowAnchor = new GPoint(5, 1); //anchors the info window to the icon

        load(); //calls the load function below
    }
}

function load() //loads controls, holds the lat/long display listener, and calls the cookie functions
{
    loadSessionCenter(); //loads center from cookie

```

```

var center = map.getCenter();
document.getElementById("message").innerHTML = center.toString();
//these two lines initialize the lat/long display

map.addControl(new GMapTypeControl());
map.addControl(new GScaleControl());
//add three controls

GEvent.addListener(map, "moveend", function()
{
    var center = map.getCenter();
    document.getElementById("message").innerHTML = center.toString();
    //gives current lat/long
});

GEvent.addListener(map, "click", function(overlay, point)
{
    clickListenerFunction(overlay, point);
    //calls function to test what click should do
});

document.getElementById("CurrentSessionName").innerHTML = c_name;
//returns session name

countCookies();
//determines the number of items and stores in items global variable
useCookies();
//begins using cookies

}

function setSessionName()
{
    //sets the current session name

    var start, end;
    start = document.cookie.indexOf("Current_C_name_that_is_being_used_by_this_program");
    //stores start and end indexes
    start = document.cookie.indexOf(" ", start);
    //finds the index of the first ' '
    end = document.cookie.indexOf(" ", start);
    //finds the index of the following
    //stores current session name as the
    c_name

    '),'
    c_name = document.cookie.substring(start+1, end);
    //stores current session name as the
    c_name

    if(c_name == "")
    //if c_name is null, a c_name must
    be given by the user
    {

```

```

c_name=prompt('Please enter your session name:', ""); //prompts the user for a session/user name

if (c_name == null || c_name == "") //if the c_name is null, set to 'default'
    c_name = "default";

storeNewSession(c_name); //calls function to store c_name in a cookie
storeSessionCenter(); //stores current location as the session center
}

}

function storeNewSession(tempCname) //stores new session name in a cookie
{
    if(checkCookieForSession(tempCname) == -1) //if the c_name is already in the cookie, do not overwrite
    {
        var exdate = new Date();
        exdate.setDate(exdate.getDate()+400); //cookie expires 400 days in the future

        document.cookie = tempCname+ "=" + "DELIMITER" + ";" + ";expires="+exdate; //since it has no cookies, it's set to null
    }

    changeSessionName(tempCname); //calls function to set current session name cookie
}

function checkCookieForSession(tempCname) //checks to see if the session name is already in the
cookie
{
    return document.cookie.indexOf(tempCname); //returns -1 if the name is not found
}

function changeSessionName(tempCname) //sets a default cookie to store the current
session name
{
    var exdate = new Date();
    exdate.setDate(exdate.getDate()+400); //cookie expires 400 days in the future

    var name = "Current_C_name_that_is_being_used_by_this_program="
    document.cookie = name + "(" + tempCname + ")," + "expires="+exdate; //full cookie string

    location.reload(true); //reloads the browser so change takes effect
}

```



```

function storesSessionCenter() //stores the current center and zoom level in the cookie
{
    var tempCookie; //variable to hold cookie info
    tempCookie = getCookie(); //gets the current session's cookie info
    getSessionCenter(); //sets global center variables to current location

    tempCookie = tempCookie.substring(tempCookie.indexOf("DELIMITER"), tempCookie.length); //stores the latitude
    tempCookie = "["+centerLat+"]"+"+centerLong+"["+centerZoom+"]"+tempCookie; //adds current
    center to the front of the cookie
    setCookie(tempCookie);
    //store information back into Cookie

    var bold = document.createElement("b"); //creates a bold text node to be used in the info window
    bold.appendChild(document.createTextNode("NEW MAP CENTER")); //Document Object Model (DOM) elements

    map.openInfoWindow(map.getCenter(), bold); //pop-up info box to show new center
}

function getSessionCenter() //gets the current center and zoom level for the session
{
    var start, end; //stores the start and end of the latitude and longitude
    var first, second; //stores the latitude and longitude
    var center;

    center = map.getCenter().toString(); //gets center and stores it as a string

    if(center != "null") //if there is a center defined
    {
        start = center.indexOf("(") + 1; //gets the start position of the latitude
        end = center.indexOf(","); //gets the end position of the latitude
        first = center.substring(start, end); //stores the latitude

        start = center.indexOf(",") + 2; //gets the start position of the longitude
        end = center.indexOf(")", start); //gets the end position of the longitude
        second = center.substring(start, end); //stores the longitude

        centerLat = first;
        centerLong = second;
        centerZoom = map.getZoom();
    }

    //these three variables are global and keep track of center zoom and position
}

```

```

    }
}

function loadSessionCenter() // loads the center and zoom level from the cookie
{
    var wholeString = getCookie(); //stores the sessions cookie in a string
    var parseStart, parseEnd=0; //holds locations within the string
    var killLoad = 0; //used as a flag to determine the end of the string
    var centerString = []; //array that stores the center and zoom level info
    var i;

    if (wholeString.length>0) //if there is a middle portion of the cookie. . .
    {
        for(i=0; i<3; i++)
        {
            parseStart=wholeString.indexOf("[",parseEnd) + 1; //finds the indexes of the elements
            parseEnd=wholeString.indexOf("]",parseStart);

            if (parseEnd < parseStart) //i.e. if parseEnd is negative, it is not a proper value
            {killLoad = 1;} //this indicates the end of the string
            else
            {
                centerString[i] = wholeString.substring(parseStart,parseEnd);

                parseStart = parseEnd + 1; //moves start forward to proper bracket
            }
        }

        if(killLoad != 1) //if the was parsed correctly, use what was parsed
        {
            centerLat = centerString[0];
            centerLong = centerString[1];
            centerZoom = parseInt(centerString[2]);
        }
        else //else, used the default location and zoom level
        {
            centerLat = 21.94304553343818; //default location and zoom level allows the majority of the world to be seen
            centerLong = 11.6015625;
            centerZoom = 2;
        }
    }
    map.setCenter(new GLatLng(centerLat, centerLong), centerZoom); //starting location and zoom level
}

```

```

function getSessionNames() //gets session names
{
    var start=0; //holds start index
    var end=0; //holds end index
    var i=0, x; //counter variables
    var tempCName;
    var test = "";

    if (document.cookie.length>0)
    {
        var cookie = document.cookie;
    }

    var cnames = []; //stores c_name(s)

    end = cookie.indexOf("="); //end of the first one

    tempCName = cookie.substring(0,end); //stores the first session name

    if(tempCName != "Current_C_name_that_is_being_used_by_this_program") //current session holder
    {
        cnames[i] = tempCName; //stores session names
        i=1;
    }

    while(start >= 0 && end >= 0) //if start or end is negative, then they are outside of the string and
    { //as long as they are positive, the function is still
        incorrect //working properly

        start = cookie.indexOf(",","end");
        end = cookie.indexOf("=", start);

        if(start >= 0 && end >= 0) //same reason as above, checks again before storing the information
        {
            tempCName = cookie.substring(start+2, end);

            if(tempCName != "Current_C_name_that_is_being_used_by_this_program") //current session holder
            {
                cnames[i] = tempCName; //stores session names for use in drop down menu
                i++;
            }
        }
    }
}

```

```

    }
    for (x=0; x<i; x++)
    {
        document.write("<option value= \""+cnames[x]+"\">"+cnames[x]);
    }
    //writes HTML for drop down menu
}

function deleteSession()
{
    //this deletes all session information
    var exdate = new Date();

    exdate.setDate(exdate.getDate()-1);
    document.cookie = c_name + "=" + "DELIMITER" + ";" + "expires="+exdate;
    //cookie "expires" yesterday
    //rewrites the cookie

    exdate.setDate(exdate.getDate()+400);
    var name = "Current_C_name_that_is_being_used_by_this_program=";
    document.cookie = name + "(" + " " + " " + " " + "expires="+exdate;
    //cookie expires 400 days in the future
    //full cookie string

    location.reload(true);
    //reloads the browser so change takes effect
}

function useCookies()
{
    //creates markers from broken down cookie
    var coord;
    //stores coordinates

    var string = [];
    for (var i=0; i<items; i++)
    {
        //store individual data elements
        //items gives number of elements, it is a global variable
        if (flag == 1)
        {
            //while flag == 1, there are still more elements to parse out
            {
                string[i] = parseString();
                //stores results of parseString() in string[i]
            }
        }
    }

    for (var i=0; i<items; i+=5)
    {
        //marker data is grouped in fives
        //the first data element is the marker ID, given by date in milliseconds
        coord = new Glatlng(string[i+1], string[i+2]);
        //coordinates are the second and third data elements
    }

    color = string[i+4];
    icon.image = "http://labs.google.com/ridefinder/images/mm_20_"+color+".png";
    //retrieves the color stored in the cookie
    //sets the color of the icon to
    match the saved color
}

```

```

createMarker(coord, string[i+3], string[i]); //this calls the function to create a marker
}

color = "red";
icon.image = "http://labs.google.com/ridefinder/images/mm_20_"+color+".png"; //sets color back to default

getPolyLines(); //get polyline information from the cookie
}

function parseString() //goes through the middle portion of cookie returned from getCookie() and
breaks it into individual data elements
{
    var wholeString = getCookie();
    var subString;
    var count = 0;

    if (wholeString.length>0) //if there is a middle portion of the cookie. . .
    {
        parseStringStart=wholeString.indexOf('"',parseStringEnd) + 1;
        parseStringEnd=wholeString.indexOf('"',parseStringStart);

        if (parseStringEnd < parseStringStart) //i.e. if parseString end is negative, it is not a proper
        value {flag = 0} //this indicates the end of the string
        else
        {
            subString = wholeString.substring(parseStringStart,parseStringEnd);
            parseStringStart = parseStringEnd + 1; //moves start forward to proper paranthesis based on
            cookie structure
        }
        return subString
    }
}

function getPolyLines() //gets the polyline data from the cookie
{
    var string = []; //store individual data elements
    var lastIndex=0; //holds the location of the last polyline set found so the same one won't be pulled
    in infinitely

```

```

if(polyItems > 0)           //if there are any polyline sets
{
    string = parsePolyLines(lastIndex);           //gets the first polyline set
    currentMarkerID = string[0];
    makeNewPoly(string[1], "false");

    for(var i=1; i<polyItems; i++)
    {
        lastIndex = parseInt(string[2]);           //get last index position
        string = parsePolyLines(lastIndex); //stores results of parsePolyLines() in string[]

        currentMarkerID = string[0];           //gets first ID, sets to currentMarkerID
        makeNewPoly(string[1], "false");           //gets second ID, calls function
    }
}

function parsePolyLines(lastIndex)           //parses the polyline information
{
    var wholeString = getCookie();
    var subString;
    var count = 0;
    var start, end;
    var string = [];

    if (wholeString.length>0)           //if there is a middle portion of the cookie. . .
    {
        start = wholeString.indexOf("{", lastIndex) + 1;           //gets the start position of the first markerID
        end = wholeString.indexOf("}", start);           //gets the end position of the markerID
        string[0] = wholeString.substring(start,end);           //stores the first markerID

        start = wholeString.indexOf(" ", end) + 1;           //gets the start position of the second markerID
        end = wholeString.indexOf("}", start);           //gets the end position of the markerID
        string[1] = wholeString.substring(start,end);           //stores the second markerID

        string[2] = end;           //stores the ending index

        return string
    }
}

function getCookie()           //gets the contents of the cookie and returns the middle, usable portion
{

```

```

if (document.cookie.length>0)
{
    c_start=document.cookie.indexOf(c_name + "=")           //looks for its name and an equal sign to start
    if (c_start!=-1)
    {
        c_start=c_start + c_name.length+1                    //ends when it finds the first semicolon
        c_end=document.cookie.indexOf(";",c_start);
        if (c_end==-1) c_end=document.cookie.length;
        return document.cookie.substring(c_start,c_end);
    }
    return null;
}

function setCookie(info)           //sets the contents of the cookies
{
    var exdate = new Date();
    exdate.setDate(exdate.getDate()+400);           //cookie expires 400 days in the future
    document.cookie = c_name+ "=" + info + ";" + ";expires="+exdate; //full cookie string
}

function countCookies()           //the function counts the number of elements in the cookie
{
    var currentInfo = getCookie();
    items = currentInfo.split("").length-1;
    polyItems = currentInfo.split("{").length-1;
    polyItems = polyItems - currentInfo.split("{b").length - currentInfo.split("{f").length + 2;
    //subtracts the curly brackets from the CSS code
}

function updateCookie(newInfo)           //stores the new marker in the cookie
{
    var currentInfo;           //gets the current contents of the cookie with the getCookie function

    currentInfo = getCookie();

    currentInfo += newInfo;           //adds the new marker information to the information already in the cookie

    setCookie(currentInfo);           //rewrites the cookie to include the new information

    items = currentInfo.split("").length-1; //updates the items global variable
}

```

```

function clickListenerFunction(marker, point) //used by mouse click listeners to determine what action is to be
taken
{
    if(marker) //if there is a marker at the given location
    {
        if(document.deleteMarkerCheckBox.deleteMarker.checked) //if the delete marker checkbox is checked
        {
            map.removeOverlay(marker); //remove the overlay from the map
            deleteOneMarker(marker.info, "true"); //delete the particular marker info from
            the cookie
        }
        else if(document.changeCommentCheckBox.changeComment.checked)
        {
            changeComments(marker.info); //changes marker comments
            map.removeOverlay(marker); //remove the overlay from the map
        }
        else if(document.makePolyLine.makePoly.checked)
        {
            makeNewPoly(marker.info, "true"); //accepts marker ID
        }
        else
        {
            if(document.makePolyLine.makePoly.checked) //if makeCheckBox is checked, but marker
            was not clicked //make a record at the click location
            {
                var newMarker = createRecord(point.toString(), ""); //with a polyline attached
                makeNewPoly(newMarker.info, "true");
            }
            else if(document.makeCheckBox.make.checked) //if makeCheckBox is checked
            {
                createRecord(point.toString(), ""); //make a record at the click location
            }
        }
    }
}

function deleteOneMarker(info, deletePoly) //deletes the information of the particular marker
{
    var parseStart, parseEnd; //used to hold string indexes for the marker
    information //stores cookie
    var tempCookie = getCookie(); //for marker locations
    var markerCookieHalf1, markerCookieHalf2; //for center location
    var start, end, centerTemp;

```